Materiał 02 - Architektura warstwowa, walidacja i obsługa błędów

Cel

W poprzednim materiale cała aplikacja mieściła się w jednym pliku `server.js`. Dla kilku endpointów to wystarcza. Dla prawdziwej aplikacji - nie.

W tym materiale przebudujemy API tak, żeby:

- każda część kodu miała **jedną odpowiedzialność**,
- **validation** była deklaratywna i powtarzalna,
- błędy były obsługiwane w **jednym miejscu**,
- API zwracało błędy w **jednym, spójnym formacie**,
- **business logic** dało się czytać (i później testować) bez Expressa.

Po jego przejściu powinieneś rozumieć:

- czym jest **middleware** i jak działa `next()`,
- czym jest architektura warstwowa: **routes → controllers → services → repositories**,
- po co oddzielamy `app.js` od `server.js`,
- jak walidować `params`, `query` i `body` przy użyciu biblioteki **Zod**,
- jak działa **global error handling** w ExpressJS 5,
- czym jest **DTO** i dlaczego nie zwracamy obiektów "prosto z bazy",
- czym jest **state machine** w logice biznesowej,
- co to jest **dependency injection** w najprostszej postaci (funkcja, która dostaje zależności jako argumenty).

Nadal **nie używamy bazy danych**. Ale przygotujemy kod tak, żeby w następnym materiale podmienić tylko jedną warstwę.

> **Wersje użyte w materiale:** Node.js 22, `express@5`, `zod@4`. Kod był uruchamiany i testowany na tych wersjach.

---

1. Problem z jednym plikiem

Spójrz na endpoint z materiału 01:

```
app.post("/api/tasks", (req, res) => {
  const { title } = req.body;

  if (!title || title.trim().length < 3) {
    return res.status(400).json({ message: "Title must contain at least 3
characters" });
  }

  const task = { id: tasks.length + 1, title: title.trim(), completed:
false };
  tasks.push(task);

  res.status(201).json(task);
});
```

W jednej funkcji mieszają się cztery różne rzeczy:

Odpowiedzialność	Fragment
obsługa HTTP	`req.body`, `res.status(201)`
validation	`if (!title ...)`
business logic	"nowe zadanie ma `completed: false`"
dostęp do danych	`tasks.push(task)`

Dopóki endpointów jest 5, da się z tym żyć. Przy 50 zaczynają się problemy:

- ta sama validation jest kopiowana w wielu miejscach,
- każdy endpoint zwraca błędy w trochę innym formacie,
- żeby przetestować regułę biznesową, trzeba uruchomić server i wysłać request,

- podmiana tablicy na bazę danych wymaga zmian w **każdym** endpoincie.

Przy okazji: `id: tasks.length + 1` to błąd. Po usunięciu zadania o ID 1 kolejne nowe zadanie dostanie ID, które już istnieje. To dobry przykład, dlaczego generowanie ID powinno być w jednym miejscu (repository, a później baza danych).

---

2. Middleware - jak naprawdę działa Express

Zanim podzielimy kod, musimy zrozumieć mechanizm, na którym zbudowany jest cały Express.

Middleware to funkcja o sygnaturze:

```
function middleware(req, res, next) {
  // ...
}
```

Express trzyma listę takich funkcji i dla każdego requestu wywołuje je **po kolei**, w kolejności rejestracji. Każdy middleware ma trzy możliwości:

1. **zakończyć request** - wysłać response (`res.json(...)`, `res.send(...)`),
2. **przekazać dalej** - wywołać `next()`,
3. **zgłosić błąd** - wywołać `next(error)`.

```
Request
   |
   v
[ requestLogger ] --next()--> [ express.json() ] --next()--> [ route handler ]
--> Response
```

Twój route handler (`app.get("/api/tasks", (req, res) => ...)`) to też middleware - po prostu zwykle nie używa `next`.

Ważne

Jeżeli middleware **ani nie wyśle odpowiedzi, ani nie wywoła** `next()`, request "zawiśnie". Klient będzie czekał aż do timeoutu.

`express.json()` to też middleware

W materiale 01 dodaliśmy:

```
app.use(express.json());
```

Ten middleware czyta strumień danych z requestu, parsuje JSON i zapisuje wynik w `req.body`. Dlatego musi być zarejestrowany **przed** endpointami, które z `req.body` korzystają.

Przykład - własny middleware logujący

```
// src/middleware/requestLogger.js
const { randomUUID } = require("node:crypto");

function requestLogger(req, res, next) {
  req.id = req.get("X-Request-Id") || randomUUID();
  res.set("X-Request-Id", req.id);

  const start = process.hrtime.bigint();

  res.on("finish", () => {
    const ms = Number(process.hrtime.bigint() - start) / 1e6;
    console.log(`[${req.id}] ${req.method} ${req.originalUrl} -> $
{res.statusCode} (${ms.toFixed(1)} ms)`);
  });

  next();
}

module.exports = requestLogger;
```

Co tu się dzieje:

- każdy request dostaje unikalne **request ID** (albo przejmuje je od klienta z nagłówka `X-Request-Id`),
- ID trafia do nagłówka odpowiedzi - klient może je podać przy zgłaszaniu błędu,
- `res.on("finish", ...)` rejestruje callback wywoływany **po wysłaniu** odpowiedzi - dopiero wtedy znamy status code,
- `next()` przekazuje request dalej.

Przykładowy log:

```
[0b6c...] POST /api/tasks -> 201 (3.4 ms)
[a91e...] PATCH /api/tasks/4/status -> 409 (1.2 ms)
```

Request ID wydaje się detalem, ale w produkcji to podstawowe narzędzie: użytkownik zgłasza "dostałem błąd", podaje ID, a Ty znajdujesz w logach dokładnie ten request.

---

3. Architektura warstwowa

Dzielimy aplikację na warstwy. Każda warstwa rozmawia **tylko z warstwą bezpośrednio pod nią**.

```
HTTP Request
     |
     v
+--------------+   wie o: URL, HTTP method, middleware
|   routes     |   NIE wie o: logice biznesowej
+--------------+
     |
     v
+--------------+   wie o: req, res, status codes, DTO
| controllers  |   NIE wie o: tym, gdzie są dane
+--------------+
     |
     v
```

```
+--------------+   wie o: regułach biznesowych
|  services    |   NIE wie o: req, res, HTTP, Expressie
+--------------+
     |
     v
+--------------+   wie o: tym, gdzie i jak przechowywane są dane
| repositories |   NIE wie o: regułach biznesowych
+--------------+
     |
     v
  Data (tablica / baza danych)
```

Najważniejsza zasada: **service nie zna Expressa**. Nie dostaje `req`, nie zwraca status code, nie woła `res.json()`. Dostaje zwykłe dane i zwraca zwykłe dane (albo rzuca błąd).

Dzięki temu:

- ten sam service może być wywołany z REST API, z CLI, z kolejki wiadomości, z testu,
- regułę biznesową przetestujesz bez uruchamiania servera,
- podmiana tablicy na bazę danych dotyka tylko repositories.

Struktura katalogów

Przykład dla aplikacji Task Manager (zadania przypisane do projektów):

```
task-manager/
├── package.json
└── src/
    ├── app.js                # konfiguracja aplikacji Express
    ├── server.js             # uruchomienie servera (listen)
    ├── errors/
    │   └── AppError.js
    ├── middleware/
    │   ├── requestLogger.js
    │   ├── validate.js
    │   ├── notFound.js
    │   └── errorHandler.js
    └── modules/
```

```
        └── tasks/
            ├── tasks.routes.js
            ├── tasks.controller.js
            ├── tasks.service.js
            ├── tasks.repository.memory.js
            ├── tasks.schemas.js
            └── tasks.mapper.js
```

Grupujemy pliki **według funkcjonalności** (`modules/tasks`), a nie według typu (`controllers/`, `services/`). Przy dziesięciu modułach od razu widać, co należy do czego.

Dlaczego `app.js` i `server.js` osobno?

```
// src/server.js
const { createApp } = require("./app");
// ...
const app = createApp(/* zależności */);
app.listen(3000, () => console.log("Server running on http://
localhost:3000"));
```

`app.js` tylko **buduje** aplikację. `server.js` ją **uruchamia**.

Po co? W materiale 05 testy integracyjne zaimportują `createApp()` i wyślą do niego requesty bez otwierania portu. Gdyby `app.listen()` było w tym samym pliku, każdy test próbowałby uruchomić server na porcie 3000.

---

4. Repository - jedyne miejsce, które dotyka danych

Na razie dane trzymamy w pamięci. Ale tym razem **zamykamy je w obiekcie** z jasno określonymi metodami:

```
// src/modules/tasks/tasks.repository.memory.js
```

```
function createInMemoryRepositories(seed = {}) {
  const state = {
    projects: structuredClone(seed.projects ?? []),
    tasks: structuredClone(seed.tasks ?? [])
  };
  let nextTaskId = Math.max(0, ...state.tasks.map(t => t.id)) + 1;

  const projectsRepository = {
    async findById(id) {
      return state.projects.find(p => p.id === id) ?? null;
    }
  };

  const tasksRepository = {
    async findById(id) {
      return state.tasks.find(t => t.id === id) ?? null;
    },
    async create(data) {
      const task = { id: nextTaskId++, description: null, deadline: null,
createdAt: new Date(), ...data };
      state.tasks.push(task);
      return task;
    },
    async updateStatus(id, status) {
      const task = state.tasks.find(t => t.id === id);
      task.status = status;
      return task;
    },
    async countByProjectAndStatus(projectId, status) {
      return state.tasks.filter(t => t.projectId === projectId && t.status ===
status).length;
    },
    async list({ status, projectId }) {
      return state.tasks.filter(t =>
        (status === undefined || t.status === status) &&
        (projectId === undefined || t.projectId === projectId)
      );
    }
  };

  return { projectsRepository, tasksRepository };
}
```

```
module.exports = { createInMemoryRepositories };
```

Kilka decyzji, które warto zrozumieć:

1. Dlaczego metody są `async`, skoro operacje na tablicy są synchroniczne?

Bo zapytania do bazy danych **będą** asynchroniczne. Jeżeli już teraz interfejs zwraca `Promise`, to w materiale 03 podmienimy implementację, a service nawet tego nie zauważy. Interfejs repository to **kontrakt** - implementacja może się zmienić.

2. Dlaczego `nextTaskId++`, a nie `tasks.length + 1`?

Licznik tylko rośnie, więc usunięcie elementu nie spowoduje duplikatu ID. W bazie danych tę rolę przejmie sekwencja (`SERIAL` / `IDENTITY`).

3. Dlaczego funkcja `createInMemoryRepositories()`, a nie zwykła tablica na poziomie modułu?

Każde wywołanie tworzy **nowy, niezależny** stan. W testach każdy test dostanie świeże dane i testy nie będą na siebie wpływać.

4. Dlaczego repository nie sprawdza, czy projekt jest zarchiwizowany?

Bo to reguła biznesowa. Repository odpowiada na pytanie "jak zapisać/odczytać", a nie "czy wolno".

---

5. Service - reguły biznesowe

Rozbudujmy domenę. Zadanie (`task`) ma teraz:

- `status`: `TODO`, `IN_PROGRESS` albo `DONE`,
- `projectId` - zadanie należy do projektu,
- opcjonalny `deadline`.

Projekt ma `status`: `ACTIVE` albo `ARCHIVED`.

Reguły biznesowe:

1. Nie można dodać zadania do nieistniejącego projektu.
2. Nie można dodać zadania do projektu `ARCHIVED`.
3. `deadline` nie może być w przeszłości.
4. Status zmienia się tylko według dozwolonych przejść (**state machine**).
5. W jednym projekcie mogą być maksymalnie **3 zadania** `IN_PROGRESS` jednocześnie (tzw. WIP limit - *work in progress*).

State machine

Zamiast pisać serię `if`-ów, opisujemy dozwolone przejścia jako dane:

```
          +-------------+
          |             v
  TODO ------> IN_PROGRESS ------> DONE
                  |
                  +------> TODO
```

```
const ALLOWED_TRANSITIONS = {
  TODO: ["IN_PROGRESS"],
  IN_PROGRESS: ["DONE", "TODO"],
  DONE: []
};
```

`DONE` to stan końcowy - nie da się z niego wyjść. Dodanie nowego statusu (np. `IN_REVIEW`) to zmiana w jednym obiekcie, a nie przeszukiwanie kodu.

Implementacja

```
// src/modules/tasks/tasks.service.js
const { NotFoundError, ConflictError, ValidationError } = require("../../
```

```
errors/AppError");

const WIP_LIMIT = 3;
const ALLOWED_TRANSITIONS = {
  TODO: ["IN_PROGRESS"],
  IN_PROGRESS: ["DONE", "TODO"],
  DONE: []
};

function createTasksService({ projectsRepository, tasksRepository, now = () =>
new Date() }) {
  async function createTask(input) {
    const project = await projectsRepository.findById(input.projectId);
    if (!project) throw new NotFoundError("Project");

    if (project.status === "ARCHIVED") {
      throw new ConflictError("Cannot add tasks to an archived project",
"PROJECT_ARCHIVED");
    }

    if (input.deadline && new Date(input.deadline) <= now()) {
      throw new ValidationError([
        { location: "body", field: "deadline", message: "Deadline must be in
the future" }
      ]);
    }

    return tasksRepository.create({ ...input, status: "TODO" });
  }

  async function changeStatus(id, newStatus) {
    const task = await tasksRepository.findById(id);
    if (!task) throw new NotFoundError("Task");

    if (task.status === newStatus) return task;

    if (!ALLOWED_TRANSITIONS[task.status].includes(newStatus)) {
      throw new ConflictError(
        `Cannot change status from ${task.status} to ${newStatus}`,
        "INVALID_STATUS_TRANSITION"
      );
    }
```

```
    if (newStatus === "IN_PROGRESS") {
      const inProgress = await
tasksRepository.countByProjectAndStatus(task.projectId, "IN_PROGRESS");
      if (inProgress >= WIP_LIMIT) {
        throw new ConflictError(`Project already has ${WIP_LIMIT} tasks in
progress`, "WIP_LIMIT_EXCEEDED");
      }
    }

    return tasksRepository.updateStatus(id, newStatus);
  }

  async function getTask(id) {
    const task = await tasksRepository.findById(id);
    if (!task) throw new NotFoundError("Task");
    return task;
  }

  async function listTasks(filters) {
    return tasksRepository.list(filters);
  }

  return { createTask, changeStatus, getTask, listTasks };
}

module.exports = { createTasksService, WIP_LIMIT };
```

Zwróć uwagę na kilka rzeczy:

Service nie zwraca status codes. Rzuca błędy domenowe (`NotFoundError`, `ConflictError`). O tym, jak zamienić je na HTTP, zdecyduje error handler (sekcja 8).

`if (task.status === newStatus) return task;` - ustawienie tego samego statusu drugi raz nie jest błędem. Operacja jest **idempotentna**: wykonana raz albo pięć razy daje ten sam efekt. To ważne, bo klient (np. przy słabym internecie) może wysłać ten sam request ponownie.

`now = () => new Date()` - aktualny czas jest **zależnością**, którą można podmienić. Brzmi dziwnie, ale wyobraź sobie test reguły "deadline nie może być w przeszłości". Jeżeli service woła `new Date()` bezpośrednio, wynik testu zależy od dnia, w którym go uruchamiasz. Gdy `now` jest parametrem, test może "zamrozić" czas.

Dependency injection

`createTasksService({ projectsRepository, tasksRepository })` to właśnie **dependency injection** w najprostszej postaci: service nie importuje repository sam, tylko **dostaje je z zewnątrz**.

```
// produkcja (od materiału 03)
createTasksService({ projectsRepository: knexProjects, tasksRepository:
knexTasks });

// test (materiał 05)
createTasksService({ projectsRepository: fakeProjects, tasksRepository:
fakeTasks, now: () => fixedDate });
```

Service nie wie i nie musi wiedzieć, z którą implementacją rozmawia. Nie potrzebujemy do tego żadnego frameworka - wystarczy funkcja z parametrami.

---

6. Validation z Zod

Gdzie kończy się validation, a zaczyna business logic?

To częste pytanie. Prosta reguła:

Validation (kształt danych)	**Business rule (stan systemu)**
Czy `title` jest stringiem o długości 3-120?	Czy projekt nie jest zarchiwizowany?
Czy `projectId` jest dodatnią liczbą całkowitą?	Czy nie przekroczono WIP limitu?
Czy `deadline` jest poprawną datą ISO 8601?	Czy przejście statusu jest dozwolone?
Da się sprawdzić **bez** dostępu do danych	Wymaga **odczytu** aktualnego stanu

Validation robimy **przed** wejściem do service. Business rules - **w** service.

Ręczna validation się nie skaluje

Endpoint tworzący zadanie przyjmuje 4 pola. Ręcznie wyglądałoby to tak:

```
if (typeof body.title !== "string" || body.title.trim().length < 3 ||
body.title.length > 120) ...
if (!Number.isInteger(body.projectId) || body.projectId <= 0) ...
if (body.deadline !== undefined && isNaN(Date.parse(body.deadline))) ...
if (body.description !== undefined && typeof body.description !==
"string") ...
```

Do tego trzeba zebrać wszystkie błędy naraz (a nie zwracać tylko pierwszy) i zwrócić je w spójnym formacie.

Zamiast tego **opisujemy kształt danych** biblioteką Zod:

```
npm install zod
```

```
// src/modules/tasks/tasks.schemas.js
const { z } = require("zod");

const idParams = z.object({
  id: z.coerce.number().int().positive()
});

const createTaskBody = z.object({
  title: z.string().trim().min(3).max(120),
  description: z.string().trim().max(1000).optional(),
  projectId: z.number().int().positive(),
  deadline: z.iso.datetime({ offset: true }).optional()
}).strict();

const changeStatusBody = z.object({
```

```
  status: z.enum(["TODO", "IN_PROGRESS", "DONE"])
}).strict();

const listTasksQuery = z.object({
  status: z.enum(["TODO", "IN_PROGRESS", "DONE"]).optional(),
  projectId: z.coerce.number().int().positive().optional()
});

module.exports = { idParams, createTaskBody, changeStatusBody,
listTasksQuery };
```

Omówmy najważniejsze elementy:

`z.coerce.number()` - wartości w `req.params` i `req.query` są **zawsze stringami** (`"5"`, a nie `5`), bo pochodzą z URL. `coerce` najpierw wywołuje `Number(value)`, a potem sprawdza wynik. Dla `"abc"` dostaniemy `NaN`, który nie przejdzie validation.

W `body` **nie używamy** `coerce` - JSON ma typy, więc klient powinien wysłać liczbę. `"projectId": "1"` to błąd klienta i zwracamy `400`.

`.trim()` - schemat nie tylko sprawdza, ale też **przekształca** dane. `"  Learn Zod  "` stanie się `"Learn Zod"`. Dlatego zawsze korzystamy z wyniku parsowania, a nie z oryginalnego `req.body`.

`z.iso.datetime({ offset: true })` - akceptuje daty ISO 8601 ze strefą czasową, np. `2026-11-10T10:00:00Z` albo `2026-11-10T10:00:00+01:00`. Bez `offset: true` zaakceptowane byłyby tylko daty z `Z` (UTC).

> W Zod 3 pisało się `z.string().datetime()`. W Zod 4 ta forma nadal działa, ale jest oznaczona jako deprecated - zalecane jest `z.iso.datetime()`. Jeżeli w tutorialu w internecie zobaczysz starszą składnię, to właśnie dlatego.

`.strict()` - odrzuca **nieznane pola**. Bez tego request:

```
{ "title": "Hack", "projectId": 1, "status": "DONE", "isAdmin": true }
```

przeszedłby validation (Zod domyślnie usuwa nieznane pola, ale ich nie zgłasza). Ze `.strict()`

klient dostanie jasny błąd `Unrecognized key`. To ochrona przed tzw. **mass assignment** - sytuacją, w której klient przemyca pola, których nie powinien ustawiać (np. `status`, `role`, `userId`).

Middleware `validate`

Schematy mamy. Teraz potrzebujemy jednego, uniwersalnego middleware, który użyje ich w dowolnym endpoincie:

```
// src/middleware/validate.js
const { ValidationError } = require("../errors/AppError");

function validate(schemas) {
  return (req, res, next) => {
    const validated = {};

    for (const part of ["params", "query", "body"]) {
      if (!schemas[part]) continue;

      const result = schemas[part].safeParse(req[part] ?? {});

      if (!result.success) {
        const details = result.error.issues.map(issue => ({
          location: part,
          field: issue.path.join("."),
          message: issue.message
        }));
        return next(new ValidationError(details));
      }

      validated[part] = result.data;
    }

    req.validated = validated;
    next();
  };
}

module.exports = validate;
```

To jest **funkcja, która zwraca middleware** (tzw. middleware factory). Dlatego używamy jej z nawiasami: `validate({ body: createTaskBody })`. Wywołanie konfiguruje middleware, a Express dostaje gotową funkcję `(req, res, next)`.

`safeParse` **vs** `parse` - `parse` rzuca wyjątek przy błędzie, `safeParse` zwraca obiekt `{ success, data }` albo `{ success, error }`. Tutaj wygodniejsze jest `safeParse`, bo chcemy sami zbudować format błędu.

`req[part] ?? {}` - w Express 5 `req.body` jest `undefined`, jeżeli request nie miał body albo nie był JSON-em. Zamieniamy to na pusty obiekt, żeby Zod zgłosił brakujące pola zamiast błędu typu.

Dlaczego `req.validated`**, a nie nadpisanie** `req.query`**?** W Express 5 `req.query` jest **getterem** - nie można przypisać do niego nowej wartości. Dlatego zwalidowane (i przekształcone) dane trzymamy w osobnym polu.

---

7. Controller i routes

Controller tłumaczy między światem HTTP a service:

```
// src/modules/tasks/tasks.controller.js
const { toTaskResponse } = require("./tasks.mapper");

function createTasksController(tasksService) {
  return {
    async list(req, res) {
      const tasks = await tasksService.listTasks(req.validated.query);
      res.json(tasks.map(toTaskResponse));
    },

    async getById(req, res) {
      const task = await tasksService.getTask(req.validated.params.id);
      res.json(toTaskResponse(task));
    },

    async create(req, res) {
      const task = await tasksService.createTask(req.validated.body);
```

```
      res.status(201).location(`/api/tasks/${task.id}
`).json(toTaskResponse(task));
    },

    async changeStatus(req, res) {
      const task = await tasksService.changeStatus(req.validated.params.id,
req.validated.body.status);
      res.json(toTaskResponse(task));
    }
  };
}

module.exports = { createTasksController };
```

Controller jest krótki i "nudny". **I tak ma być.** Jeżeli w controllerze pojawia się `if` dotyczący reguły biznesowej, to znak, że kod trafił do złej warstwy.

Nagłówek `Location` przy `201 Created` to dobra praktyka REST: mówi klientowi, pod jakim adresem znajdzie nowo utworzony zasób.

Zauważ też, że **nie ma tu** `try/catch`. Wyjaśnienie w sekcji 8.

Routes łączą URL, middleware i controller:

```
// src/modules/tasks/tasks.routes.js
const { Router } = require("express");
const validate = require("../../middleware/validate");
const { idParams, createTaskBody, changeStatusBody, listTasksQuery } =
require("./tasks.schemas");

function createTasksRouter(controller) {
  const router = Router();

  router.get("/", validate({ query: listTasksQuery }), controller.list);
  router.get("/:id", validate({ params: idParams }), controller.getById);
  router.post("/", validate({ body: createTaskBody }), controller.create);
  router.patch("/:id/status", validate({ params: idParams, body:
changeStatusBody }), controller.changeStatus);
```

```
  return router;
}

module.exports = { createTasksRouter };
```

`Router()` to "mini aplikacja" Express. Ścieżki są **względne** - `"/"` zostanie zamontowane pod `/api/tasks` w `app.js`.

Do jednej ścieżki można podpiąć **wiele** middleware. Express wywoła je po kolei: najpierw `validate(...)`, a jeżeli ten wywoła `next()`, dopiero wtedy controller.

Dlaczego `PATCH /api/tasks/:id/status`, a nie `PATCH /api/tasks/:id`?

Zmiana statusu to nie jest zwykła edycja pola - uruchamia reguły (state machine, WIP limit). Osobny endpoint czyni tę operację **jawną** w API i pozwala ją osobno walidować oraz (w materiale 04) osobno autoryzować.

---

8. Global error handling

Klasy błędów

Zamiast zwracać `res.status(404)` w wielu miejscach, definiujemy **hierarchię błędów**:

```
// src/errors/AppError.js
class AppError extends Error {
  constructor(message, statusCode, code, details) {
    super(message);
    this.name = this.constructor.name;
    this.statusCode = statusCode;
    this.code = code;
    this.details = details;
  }
}
```

```
class ValidationError extends AppError {
  constructor(details) {
    super("Validation failed", 400, "VALIDATION_ERROR", details);
  }
}

class NotFoundError extends AppError {
  constructor(resource) {
    super(`${resource} not found`, 404, "NOT_FOUND");
  }
}

class ConflictError extends AppError {
  constructor(message, code = "CONFLICT") {
    super(message, 409, code);
  }
}

module.exports = { AppError, ValidationError, NotFoundError, ConflictError };
```

Każdy błąd niesie trzy informacje:

- `statusCode` - dla protokołu HTTP,
- `code` - **stały, maszynowo czytelny** identyfikator (`WIP_LIMIT_EXCEEDED`),
- `message` - opis dla człowieka.

Po co `code`, skoro jest `message`? Frontend nie powinien porównywać tekstów (`if (message === "Project already has 3 tasks in progress")`). Tekst może się zmienić albo zostać przetłumaczony. `code` jest kontraktem API.

Error-handling middleware

Express rozpoznaje middleware obsługujący błędy po **czterech parametrach**:

```
// src/middleware/errorHandler.js
const { AppError } = require("../errors/AppError");

function errorHandler(err, req, res, next) {
```

```
  if (err instanceof AppError) {
    return res.status(err.statusCode).json({
      error: {
        code: err.code,
        message: err.message,
        details: err.details,
        requestId: req.id
      }
    });
  }

  if (err.type === "entity.parse.failed") {
    return res.status(400).json({
      error: { code: "INVALID_JSON", message: "Request body is not valid
JSON", requestId: req.id }
    });
  }

  console.error(`[${req.id}]`, err);

  res.status(500).json({
    error: { code: "INTERNAL_ERROR", message: "Unexpected error", requestId:
req.id }
  });
}

module.exports = errorHandler;
```

Ważne

Parametr `next` musi być w sygnaturze, **nawet jeżeli go nie używasz**. Express sprawdza liczbę parametrów funkcji (`fn.length === 4`). Jeżeli usuniesz `next`, Express potraktuje funkcję jak zwykły middleware i nigdy nie przekaże do niej błędu.

Trzy przypadki w handlerze:

1. `AppError` - błąd, który przewidzieliśmy. Zwracamy jego status i kod.
2. `entity.parse.failed` - błąd rzucany przez `express.json()`, gdy body nie jest poprawnym JSON-em (np. `{bad json`). Bez tej obsługi klient dostałby `500`, choć to ewidentnie jego błąd.
3. **Wszystko inne** - błąd, którego się nie spodziewaliśmy (bug, `undefined is not a function`,

padnięta baza). Logujemy **pełne** szczegóły po stronie servera, ale klientowi zwracamy **ogólny** komunikat.

Dlaczego nie zwracamy klientowi `err.message` i `err.stack` przy `500`? Stack trace zdradza strukturę katalogów, nazwy bibliotek i ich wersje, czasem fragmenty zapytań SQL. To informacje przydatne dla atakującego. Klient dostaje `requestId` - to wystarczy, żeby znaleźć szczegóły w logach.

Jak błąd trafia do error handlera?

Są dwie drogi:

```
// 1. Jawnie - przez next(error)
return next(new ValidationError(details));

// 2. Przez throw w async handlerze (Express 5)
async create(req, res) {
  const task = await tasksService.createTask(req.validated.body); // service
rzuca ConflictError
  // ...
}
```

W Express 5, jeżeli async route handler zwróci odrzucony `Promise` (czyli `throw` wewnątrz `async` funkcji), Express **sam** wywoła `next(err)`. Dlatego controllery nie potrzebują `try/catch`.

> **Uwaga na Express 4.** W Express 4 ten mechanizm nie istniał. Błąd w `async` handlerze kończył się `UnhandledPromiseRejection`, a request wisiał do timeoutu. Trzeba było pisać `try { ... } catch (err) { next(err); }` w każdym handlerze albo używać pakietu `express-async-errors`. Jeżeli w tutorialu widzisz takie konstrukcje - to kod pisany pod Express 4. `npm install express` instaluje dziś wersję 5.

404 dla nieznanych ścieżek

Domyślnie Express na nieznaną ścieżkę zwraca stronę HTML `Cannot GET /api/nope`. W API chcemy JSON w tym samym formacie co inne błędy:

```
// src/middleware/notFound.js
const { NotFoundError } = require("../errors/AppError");

module.exports = (req, res, next) => next(new NotFoundError(`Route ${req.method} ${req.originalUrl}`));
```

Ten middleware rejestrujemy **po wszystkich routes**. Jeżeli request do niego dotarł, to znaczy, że żaden route go nie obsłużył.

---

9. DTO - nie zwracamy modelu wewnętrznego

DTO (*Data Transfer Object*) to obiekt opisujący dane przesyłane przez API. Oddzielamy go od modelu wewnętrznego, czyli tego, jak dane są przechowywane.

```
// src/modules/tasks/tasks.mapper.js
function toTaskResponse(task) {
  return {
    id: task.id,
    title: task.title,
    description: task.description ?? null,
    status: task.status,
    projectId: task.projectId,
    deadline: task.deadline ? new Date(task.deadline).toISOString() : null,
    createdAt: new Date(task.createdAt).toISOString()
  };
}

module.exports = { toTaskResponse };
```

Po co, skoro na razie obiekty wyglądają prawie tak samo?

1. **Bezpieczeństwo.** W materiale 04 model użytkownika będzie zawierał `passwordHash`. Jeżeli zwracasz `res.json(user)`, to prędzej czy później hash hasła wycieknie w odpowiedzi. Mapper

zwraca **tylko** pola wymienione jawnie (whitelist).

2. **Stabilny kontrakt.** W bazie kolumna będzie się nazywać `created_at`, a w API `createdAt`. Zmiana nazwy kolumny nie może zepsuć frontendu.
3. **Spójny format.** Daty zawsze jako ISO 8601 w UTC, brakujące wartości zawsze jako `null` (a nie raz `null`, raz brak pola).

Mamy więc dwa rodzaje DTO:

- **request DTO** - opisany schematem Zod (co klient **może** przysłać),
- **response DTO** - opisany mapperem (co klient **dostanie**).

---

10. Składamy wszystko razem

```
// src/app.js
const express = require("express");
const requestLogger = require("./middleware/requestLogger");
const notFound = require("./middleware/notFound");
const errorHandler = require("./middleware/errorHandler");
const { createTasksService } = require("./modules/tasks/tasks.service");
const { createTasksController } = require("./modules/tasks/tasks.controller");
const { createTasksRouter } = require("./modules/tasks/tasks.routes");

function createApp({ projectsRepository, tasksRepository }) {
  const app = express();

  app.use(requestLogger);
  app.use(express.json());

  const tasksService = createTasksService({ projectsRepository,
tasksRepository });
  const tasksController = createTasksController(tasksService);
  app.use("/api/tasks", createTasksRouter(tasksController));

  app.use(notFound);
  app.use(errorHandler);

  return app;
```

```
}

module.exports = { createApp };
```

```
// src/server.js
const { createApp } = require("./app");
const { createInMemoryRepositories } = require("./modules/tasks/
tasks.repository.memory");

const repositories = createInMemoryRepositories({
  projects: [
    { id: 1, name: "Website", status: "ACTIVE" },
    { id: 2, name: "Old CRM", status: "ARCHIVED" }
  ]
});

const app = createApp(repositories);

app.listen(3000, () => console.log("Server running on http://
localhost:3000"));
```

Kolejność `app.use` ma znaczenie:

```
requestLogger  -> musi być pierwszy, żeby zmierzyć czas całego requestu
express.json() -> przed routes, bo routes czytają req.body
routes         -> właściwa obsługa
notFound       -> po routes - łapie to, czego żaden route nie obsłużył
errorHandler   -> zawsze ostatni
```

`server.js` to jedyne miejsce, w którym "wiemy", że dane są w pamięci. To tak zwany **composition root** - punkt, w którym składamy aplikację z konkretnych implementacji. W materiale 03 zmienimy tylko ten plik i dopiszemy nowe repository.

---

11. Jak to działa w praktyce

Request z błędnymi danymi:

```
POST /api/tasks
Content-Type: application/json

{ "title": "x", "projectId": "1", "isAdmin": true }
```

Response `400 Bad Request`:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Validation failed",
    "details": [
      { "location": "body", "field": "title", "message": "Too small: expected
string to have >=3 characters" },
      { "location": "body", "field": "projectId", "message": "Invalid input:
expected number, received string" },
      { "location": "body", "field": "", "message": "Unrecognized key:
\"isAdmin\"" }
    ],
    "requestId": "03fda03c-5345-4689-b866-a6902d7da099"
  }
}
```

Klient dostaje **wszystkie** problemy naraz, a nie tylko pierwszy. Formularz na frontendzie może podświetlić każde błędne pole.

Request łamiący regułę biznesową (czwarte zadanie `IN_PROGRESS`):

```
PATCH /api/tasks/4/status
Content-Type: application/json
```

```
{ "status": "IN_PROGRESS" }
```

Response `409 Conflict`:

```
{
  "error": {
    "code": "WIP_LIMIT_EXCEEDED",
    "message": "Project already has 3 tasks in progress",
    "requestId": "27450492-05ab-4ac4-908a-22413d3ea750"
  }
}
```

Dane były **poprawne** (status `IN_PROGRESS` istnieje), ale **stan systemu** nie pozwala na operację. Dlatego `409`, a nie `400`.

Przepływ requestu

```
PATCH /api/tasks/4/status
  |
  v
requestLogger          -> nadaje req.id
  |
  v
express.json()         -> req.body = { status: "IN_PROGRESS" }
  |
  v
validate(...)          -> req.validated = { params: { id: 4 }, body: { status:
"IN_PROGRESS" } }
  |
  v
controller.changeStatus
  |
  v
tasksService.changeStatus(4, "IN_PROGRESS")
  |    tasksRepository.findById(4)
  |    sprawdzenie state machine
  |    tasksRepository.countByProjectAndStatus(1, "IN_PROGRESS") -> 3
```

```
  |    throw ConflictError("WIP_LIMIT_EXCEEDED")
  |
  v
(Express 5 łapie odrzucony Promise -> next(err))
  |
  v
errorHandler           -> 409 + JSON
  |
  v
requestLogger (finish) -> "[...] PATCH /api/tasks/4/status -> 409 (1.2 ms)"
```

---

12. Podsumowanie

```
routes        -> URL + HTTP method + middleware (validate)
controllers   -> req/res <-> service, status codes, DTO
services      -> reguły biznesowe, rzucają błędy domenowe
repositories  -> jedyne miejsce, które dotyka danych
errorHandler  -> jedyne miejsce, które zamienia błędy na HTTP
```

Najważniejsze rzeczy:

- **Middleware** to łańcuch funkcji `(req, res, next)`. Kolejność rejestracji ma znaczenie.
- **Service nie zna Expressa.** Dostaje dane, zwraca dane albo rzuca błąd.
- **Validation** sprawdza kształt danych (Zod). **Business rules** sprawdzają stan systemu (service).
- `400` = złe dane, `404` = brak zasobu, `409` = poprawne dane, ale stan systemu nie pozwala.
- **Jeden format błędów** w całym API, ze stałym `code` dla frontendu.
- Nigdy nie zwracamy klientowi stack trace. Zwracamy `requestId`.
- **DTO** chroni przed wyciekiem pól i oddziela API od sposobu przechowywania danych.
- Zależności (repositories, czas) **przekazujemy z zewnątrz** - dzięki temu podmienimy je w materiale 03 (baza danych) i 05 (testy).